

# Mikroişlemciler Dersi – Ödev Raporu

## Kaçan Nesne Takibi Oyunu (8086 Assembly / EMU8086)

---

|                 |                        |
|-----------------|------------------------|
| <b>Ad Soyad</b> | Esmâ Sude Karadağ      |
| <b>Numara</b>   | 170424841              |
| <b>Ders</b>     | Mikroişlemciler        |
| <b>Konu</b>     | 8086 Assembly ile Oyun |
| <b>Ortam</b>    | EMU8086                |
| <b>Dosya</b>    | kacan_nesne_oyunu.asm  |

---

### 1. Projenin Amacı

Bu ödevde 8086 Assembly kullanarak EMU8086 üzerinde çalışan basit bir oyun yazmayı hedefledim. Oyunun mantığı şu: ekranda kırmızı bir kare (blok karakter) rastgele konumlara ışınlanıyor, oyuncu da mouse ile bu kareye tıklamaya çalışıyor. Her yakalamada nesne biraz küçülüyor, dolayısıyla tıklamak zorlaşıyor. 30 saniye bitince oyun duruyor ve skor gösteriliyor.

Projeyi yaparken özellikle interrupt kullanımını, timer ile zamanlama yapmayı ve mouse okumayı öğrenmiş oldum. Ayrıca COM formatında tek segment ile çalışmanın kısıtlamalarını da tecrübe ettim.

---

### 2. Oyunun Kuralları

- Oyun başlayınca ekrana bir açılış mesajı geliyor, herhangi bir tuşa basılınca oyun alanı açılıyor.
- Kırmızı blok ( karakteri) belirli aralıklarla ekranda rastgele bir yere ışınlanıyor.
- Fare ile bu bloğa tıklarsak skor 1 artıyor ve nesne %10 küçülüyor.
- Nesne en küçük hali olan 1x1'e ulaştığında ekranda "KAZANDIN!" yazısı çıkıyor.
- Her 5 saniyede nesnenin ışınlanma hızı artıyor (daha az tick bekleniyor).
- 30 saniye dolunca oyun bitiyor, toplam skor ekrana basılıyor.

---

### 3. Kullandığım Interrupt'lar

#### INT 10h (Video)

Ekrana ilgili her şeyi bu interrupt ile yaptım. Mod 3 (80x25 text mode) ayarlıyorum, imleci konumlandırıyorum, karakter yazıyorum. Nesne çizmek

için AH=09h fonksiyonunu kullandım — bu fonksiyon aynı karakteri CX kadar tekrar yazabiliyor, bu sayede döngü kurmadan tek seferde satır çizebiliyorum.

İmleci gizlemek için AH=01h'da CH'nin 5. bitini set ediyoruz (CH=20h). Bunu yapmassak oyun sırasında imleç titreyip duruyor.

### INT 21h (DOS)

String yazdırmak için AH=09h (dolar işaretine kadar yazar), tek karakter yazdırmak için AH=02h, programı kapatmak için AH=4Ch kullandım. SAYI\_YAZ rutininde rakamları teker teker AH=02h ile basıyorum.

### INT 1Ah (Saat)

Zamanlama için kullandım. AH=00h ile çağırınca DX'te anlık tick sayısı dönüyor. BIOS saati saniyede yaklaşık 18.2 kez artıyor, ben de 18 tick'i 1 saniye kabul ettim. Süre sayacı ve hareket hızı kontrolü bu tick farkına bakılarak yapılıyor.

### INT 33h (Mouse)

AX=0 ile mouse varlığını kontrol ediyorum. AX=1 ile imleci gösteriyorum. AX=3 ile mouse'un X/Y piksel koordinatlarını ve buton durumunu okuyorum. Oyun sonunda AX=2 ile mouse imlecini gizliyorum.

---

## 4. Değişkenler

Programın başında JMP BASLA ile atlanan bir bölümde değişkenleri tanımladım:

| Değişken           | Boyut | Ne İşe Yarıyor                                                           |
|--------------------|-------|--------------------------------------------------------------------------|
| nesne_x, nesne_y   | DW    | Nesnenin ekrandaki satır/sütun konumu                                    |
| nesne_boy          | DW    | Nesnenin boyutu, başta 8                                                 |
| nesne_dx, nesne_dy | DW    | Hareket yönü (artık doğrudan kullanılmıyor ama YON_DEGISTIR'de hala var) |
| skor               | DW    | Kaç kez yakalandı                                                        |
| sure               | DW    | Kalan saniye, 30'dan geriye sayıyor                                      |
| son_san, son_har   | DW    | Zamanlayıcı referans tick değerleri                                      |

| Değişken | Boyut | Ne İşe Yarıyor                                                          |
|----------|-------|-------------------------------------------------------------------------|
| oyun_dur | DB    | 1 olunca oyun bitiyor                                                   |
| har_hz   | DW    | Kaç tick'te bir ışımlansın<br>(3'ten başlıyor,<br>azaldıkça hızlanıyor) |
| tohum    | DW    | Rastgele sayı üretici<br>için seed                                      |
| hiz_say  | DW    | 5 saniyede bir hızlanma<br>kontrolü için sayaç                          |
| fare_var | DB    | Mouse var mı yok mu                                                     |
| tik_now  | DW    | O anki tick değeri                                                      |

Hepsini DW veya DB olarak tanımladım. COM formatında hepsi aynı segmentte duruyor (DS = CS).

## 5. Programın Genel Akışı

```

Program baslar
|
|  v
Giris ekranini goster
|
Tus bekle (INT 16h) --> tus kodu tohum'a yazilir
|
Ekrani temizle, mouse'u baslat
|
Sistem saatini oku, zamanlayicilari kur
|
Nesneyi rastgele bir yere koy, ciz
|
+-----> OYUN DONGUSU <-----+
|           |
|   oyun_dur = 1 mi?   |
|           |
|   hayir             |
|           |
|   tick oku          |
|   sure kontrolu yap  |
|   hareket kontrolu yap |
|   fare kontrolu yap  |
|           |
|           +-----+
|

```

```
v (oyun_dur = 1)
Sonuc ekranini goster
Tus bekle, cik
```

---

## 6. Alt Programların Açıklaması

**ZAMAN\_GUNCELLE:** Her döngüde çağrılıyor. Son güncellemenin üzerinden 18 tick geçtiyse süreyi 1 azaltıyor. Sure 0 olunca oyun\_dur'u 1 yapıyor. Ayrıca her 5 saniyede har\_hz'yi 1 azaltarak nesnenin daha sık ışınlanmasını sağlıyor.

**HAREKET\_KONTROL:** Son hareketten beri har\_hz kadar tick geçtiyse NESNE\_SIL → NESNE\_HAREKET → NESNE\_CIZ sırasıyla çağırıyor. Geçmediyse bir şey yapmıyor.

**NESNE\_SIL:** Nesnenin kapladığı alana boşluk karakteri (ASCII 32) yazıyor. SI sayacıyla satır satır ilerliyor, her satırda nesne\_boy kadar boşluk basıyor.

**NESNE\_CIZ:** NESNE\_SIL'in aynısı ama boşluk yerine ASCII 219 ( ) karakterini kırmızı renkte (BL=0Ch) yazıyor.

**NESNE\_HAREKET:** Eskiden dx/dy ile adım adım hareket ediyordu, şimdi her çağrıda RASTGELE ile yeni X ve Y üretip nesneyi oraya ışınıyor. X için mod 65 + 3, Y için mod 16 + 2 alıyorum ki nesne ekran dışına çıkmasın.

**YON\_DEGISTIR:** nesne\_dx'i NEG ile tersine çeviriyor, %50 ihtimalle nesne\_dy'yi de çeviriyor. Şu anki kodda hareket lineer olmadığı için doğrudan kullanılmıyor ama başlangıçta ve yakalama sonrasında çağrılıyor.

**RASTGELE:** Basit bir LCG (Linear Congruential Generator). Formülü:  $tohum = tohum * 25173 + 13849$ . 16-bit taşma zaten mod 65536 etkisi yapıyor. Gerçek rastgelelik değil ama EMU8086 için yeterli.

Tohum değerini oyun başında INT 16h'den dönen tuş kodundan alıyorum, üstüne INT 1Ah'dan gelen timer değerini ADD ile ekliyorum. Böylece her çalıştırmada farklı bir seri üretiyor.

**FARE\_KONTROL:** Bu kısmı yazmak biraz uğraştırdı. INT 33h ile okunan fare koordinatları piksel cinsinden geliyor ama benim nesne konumum text koordinatında. Her karakter 8 piksel olduğu için text koordinatını 8 ile çarpmam gerekiyor. SHR komutu EMU8086'da bazen sorun çıkardığı için çarpma işlemi üç kez ADD AX,AX yaparak yaptım ( $2^2=8$ ).

Dört sınır kontrol ediliyor: sol, sağ, üst, alt. Fare dört sınırın da içindeyse yakalama gerçekleşiyor. Yakalanınca skor artıyor, nesne %10 küçülüyor (boy \* 9 / 10), yeni rastgele konum veriliyor. Bir de sol buton bırakılana kadar bekliyorum, yoksa bir tıklamada birden fazla sayılıyor.

**BILGI\_GOSTER:** Ekranın 0. satırına skor ve süre bilgisini yazıyor. Nesne boyutu 1'e düşüyse 1. satıra "KAZANDIN!" mesajı basıyor.

**SAYI\_YAZ:** AX'teki sayıyı 10'a böle böle kalanları stack'e atıyor, sonra ters sırada çekip ASCII'ye çevirip (30h ekleyerek) ekrana yazıyor. Klasik bir yöntem.

---

## 7. Karşılaştığım Sorunlar ve Çözümleri

**Nesne hep aynı yerde başlıyordu:** EMU8086'da INT 1Ah her çalıştırmada sıfırdan sayıyor. Tohum olarak sadece timer kullanınca hep aynı değeri üretiyordu. Çözüm olarak INT 16h'den dönen tuş kodunu (AX) tohuma ilk değer olarak atadım, timer değerini de üstüne ADD ile ekledim. Böylece ne zaman ve hangi tuşa basarsan farklı bir seed oluyor.

**Ektranda artık izler kalıyordu:** Nesne hareket edince eski konumda kırmızı bloklar kalıyordu. NESNE\_SIL prosedürünü hareket öncesinde çağırarak çözdüm.

**Fare tıklaması çift sayılıyordu:** Sol buton basılı kaldığı sürece her döngüde yakalama sayılıyordu. FK\_BK etiketindeki döngü ile buton bırakılana kadar bekleterek çözdüm.

**DIV overflow hatası:** hiz\_say'ı 5'e bölerken DX'i sıfırlamayı unutmuştum, Division Overflow alıyordum. Her DIV'den önce MOV DX, 0 ekledim.

---

## 8. Tasarım Tercihleri

**Timer tabanlı döngü:** Klasik LOOP ile gecikme döngüsü yapabiliirdim ama o zaman işlemci hızına bağımlı olurdu. INT 1Ah ile tick sayarak zamanlama yapınca hem fare girişine anında cevap verebiliyorum hem de farklı hızlardaki bilgisayarlarda aynı oyun hızını elde ediyorum.

**COM formatı:** .EXE yerine .COM tercih ettim çünkü EMU8086'da daha kolay derleniyor. Segment ayarlaması sadece MOV AX, CS / MOV DS, AX ile halloluyor. Tek dezavantajı 64KB sınırı ama bu oyun için fazlasıyla yeterli.

**SHR yerine ADD:** SHR AX, 3 yazmak yerine üç kere ADD AX, AX yazdım. EMU8086'nın bazı versiyonlarında SHR ile ilgili uyumluluk sorunları olabiliyor, bu yüzden güvenli tarafta kaldım.

**WORD PTR/BYTE PTR kullanmadım:** EMU8086'da bu direktifler bazen beklenmeyen davranış gösteriyor. Bunun yerine register boyutuna göre doğrudan erişim yaptım (MOV AL, [degisken] gibi).

---

## 9. Test

| Ne test ettim            | Olması gereken                | Oldu mu |
|--------------------------|-------------------------------|---------|
| Mouse olmadan çalıştırma | fare kontrolü atlanmalı       | Evet    |
| Nesneye tıklama          | skor artmalı, nesne küçülmeli | Evet    |
| 30 sn bekleme            | oyun bitmeli                  | Evet    |
| Boyut 1x1'e ulaşma       | KAZANDIN yazmalı              | Evet    |
| Hızlanma (5 sn sonra)    | har_hız azalmalı              | Evet    |

Bilinen kısıtlar: - Mouse sürücüsü yoksa oyun oynanamıyor (klavye alternatifi yok). - Tick sayacı 16-bit olduğu için gece yarısı sıfırlanmasında sorun olabilir ama oyun 30 saniye sürdüğü için pratikte fark etmez. - Text modunda en küçük birim 1 karakter olduğundan nesne 1x1'den daha küçük olamıyor.

## 10. Sonuç

Bu proje sayesinde 8086 Assembly'nin register yapısını, interrupt mekanizmasını ve bellek modelini uygulamalı olarak öğrendim. Özellikle timer ile zamanlama yapmak ve mouse koordinatlarını text koordinatlarına çevirmek en çok vakit harcadığım kısımlar oldu. Sınırlı bir ortamda bile düzgün çalışan bir oyun yazılabildiğini görmek güzel oldu.

## Ekler

### Derleme ve Çalıştırma

1. EMU8086'yı aç
2. `kacan_nesne_oyunu.asm` dosyasını yükle
3. Compile (veya F5) ile derle
4. Run ile çalıştır
5. Mouse'un emülatörde aktif olduğundan emin ol

### Dosyalar

```
mikroislemciler odevi/  
  kacan_nesne_oyunu.asm      - Ana kod (yorumsuz)  
  kacan_nesne_aciklamali.asm - Aciklamali versiyon  
  rapor.md                  - Bu rapor
```